WebMirage

WebMirage - Wanjing Han, Levi Taiji Li, Mu Zhang, Yue Jiang, Guanhong Tao, GitHub.

- Published: date not stated
- Original: <https://github.com/MoonTea0416/WebMirage>
- Preserved from: https://github.com/MoonTea0416/WebMirage (github-repository-api) on 2026-10-21
- Repository commit: 8238326116896d040e131bc516898209d2a361c2
- Licence: see the repository

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

> UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

> **Repository reading copy.** Created from documentation in MoonTea0416/WebMirage, pinned to commit 823832611689. GitHub navigation and file listings are omitted. This is selected documentation; repository code is never checked out, built or run.

`README.md`

View original document

WebMirage

Adversarial Images Hijack Web Agents from Visual Grounding to Browser Execution

Wanjing Han, Levi Taiji Li, Mu Zhang, Yue Jiang, Guanhong Tao — University of Utah

WebMirage is a red-teaming framework for vision-grounded web agents. It crafts localized adversarial perturbations on attacker-controlled webpage images (e.g., a product thumbnail or a profile photo) that cause the agent to select the attacker's element and execute the corresponding browser action, across varying webpage renderings. The attack is formulated end-to-end, from visual grounding to browser execution, rather than at model inference alone.

Two components make this work:

- **Role-slot abstraction and webpage recomposition** — models the competition between the attacker's element and its companions by recomposing pages with varying

companion sets and slot positions, so the perturbation generalizes to unseen renderings.

- **Execution-aligned supervision** — a CodeQL dataflow analysis of the agent's post-processing code identifies which output tokens actually determine the executed action, and the optimization is supervised on those tokens only.

The framework is evaluated against SeeAct and VisualWebArena agents across six VLM backbones (LLaVA, MiniCPM-o, Phi-3 Vision, and others). See the paper for the full evaluation.

Repository Layout

```
.
├── src/            # Perturbation optimization, one subdirectory per backbone
│   ├── LLaVA/
│   ├── MiniCPM-o/
│   └── Phi-3-vision/
├── scripts/        # Training / evaluation launch scripts, one subdirectory per backbone
├── dataflow/       # CodeQL query pack for agent dataflow analysis (DataflowAnalysis.ql)
├── data/           # Recomposed training / test screenshots and conversation templates
└── test/           # Pre-generated clean and perturbed samples for quick verification
```

Prerequisites

The base models must be set up manually before using this framework.

Because the backbones have conflicting library requirements, each needs its own environment and locally downloaded weights.

1. MiniCPM-o

- **Repo:** OpenBMB/MiniCPM-o
- **Setup:** Clone the repo, create a dedicated environment (e.g., `conda create -n minicpm`), and install dependencies.
- **Weights:** Download the official weights locally and record the path.

2. LLaVA

- **Repo:** haotian-liu/LLaVA
- **Setup:** Clone the repo and create a dedicated environment (e.g., `conda create -n llava`).
- **Weights:** Download LLaVA-v1.5/v1.6 weights locally and record the path.

3. Phi-3 Vision

- **Repo:** microsoft/Phi-3-vision-128k-instruct
- **Setup:** Install the compatible `transformers` version in a dedicated environment.
- **Weights:** Download the `Phi-3-vision-128k-instruct` weights locally.

Quick Verification (Phi-3 Vision)

A minimal test runs inference on both clean and perturbed screenshots and compares the agent's selections side by side.

> **Note:** This test requires Phi-3 Vision (see Prerequisites §3). The provided perturbed images were optimized for Phi-3; test data for other backbones can be generated with the training scripts in `scripts/`.

Test Data

Pre-generated test cases are located in `test/`:

```
test/
├── clean_fix_slot/         # Clean screenshots, target in a fixed slot position
├── clean_multi_slot/       # Clean screenshots, target across multiple slot positions
├── perturbed_fix_slot/     # Perturbed screenshots (ε=16/255), target in a fixed slot
├── perturbed_multi_slot/   # Perturbed screenshots (ε=16/255), target across multiple
slots
├── conv.json               # Conversation data (system prompt, user query, grounding
prompt)
└── verify.py               # Runs inference on clean vs. perturbed and compares results
```

The `fix_slot` cases test companion variation (target position fixed, surrounding elements vary); the `multi_slot` cases test companion variation and positional shift simultaneously.

Running the Test

```
conda activate phi3vision

# Fixed slot position
python test/verify.py \
    --model-path /path/to/Phi-3-vision-128k-instruct \
    --conv-path test/conv.json \
    --clean-image test/clean_fix_slot/sample_0.png \
    --perturbed-image test/perturbed_fix_slot/sample_0.png

# Varying slot positions
python test/verify.py \
    --model-path /path/to/Phi-3-vision-128k-instruct \
    --conv-path test/conv.json \
    --clean-image test/clean_multi_slot/sample_0.png \
    --perturbed-image test/perturbed_multi_slot/sample_0.png
```

Expected Output

The script prints a comparison table showing, for each test case, the element selected under clean vs. perturbed input. On clean screenshots the agent should select the best-matching candidate; on perturbed screenshots it should consistently select the attacker-controlled candidate.

Optimizing a Perturbation (LLaVA Example)

With a dedicated Conda environment for each backbone in place, the workflow below uses LLaVA as the example.

1. Clone LLaVA

The scripts reference the official LLaVA code from your **home directory** (`~/`):

```
cd ~
git clone https://github.com/haotian-liu/LLaVA.git
```

2. Configure the Training Script

Open `scripts/LLaVA/train_with_mask.sh` and set each path (model weights, training data, conversation template, mask, output directory) to match your local setup.

3. Run

```
bash scripts/LLaVA/train_with_mask.sh
```

Data Format

Training data lives under `data/<scenario>/`, with recomposed screenshots and a `train_samples.json` conversation file. The image-token placeholder in the conversation file must match the target backbone:

Backbone	Placeholder
MiniCPM-o	`(<image>./</image>)\n`
LLaVA	`<image>\n`
Phi-3	`` `<\ ``

End-to-End Agent Evaluation

End-to-end evaluation requires the target agent frameworks.

SeeAct

- **Repo:** OSU-NLP-Group/SeeAct
- **Setup:** Clone the repo and follow the official installation instructions.
- **Usage:** Evaluation uses the client-side asset replacement protocol described in the paper. See `scripts/seeact/`.

VisualWebArena

- **Repo:** web-arena-x/visualwebarena
- **Setup:** Follow the official instructions to set up the sandbox, including deploying the OneStopShop shopping site.
- **Usage:** See `scripts/visualwebarena/` for evaluation scripts with both Accessibility Tree and Set-of-Mark observation modes.

Dataflow Analysis

`dataflow/DataflowAnalysis.ql` is the CodeQL query used to trace which model-output tokens flow into the executed browser action in an agent's post-processing code. Run it against the target agent's source with the CodeQL CLI using the pack definition in `dataflow/codeql-pack.yml`.

Ethics

All public-website experiments in the paper use client-side asset replacement: adversarial images are substituted in the agent's local browser view only, and no content is uploaded to or modified on any third-party site. Please use this framework for defensive research only.

Citation

```
@article{han2026webmirage,
  title   = {Adversarial Images Hijack Web Agents from Visual Grounding to Browser Execution},
  author  = {Han, Wanjing and Li, Levi Taiji and Zhang, Mu and Jiang, Yue and Tao, Guanhong},
  journal = {arXiv preprint arXiv:XXXX.XXXXX},
  year    = {2026}
}
```

License

MIT. See LICENSE.

Archived from https://github.com/MoonTea0416/WebMirage. Rendered offline from the local Markdown copy.